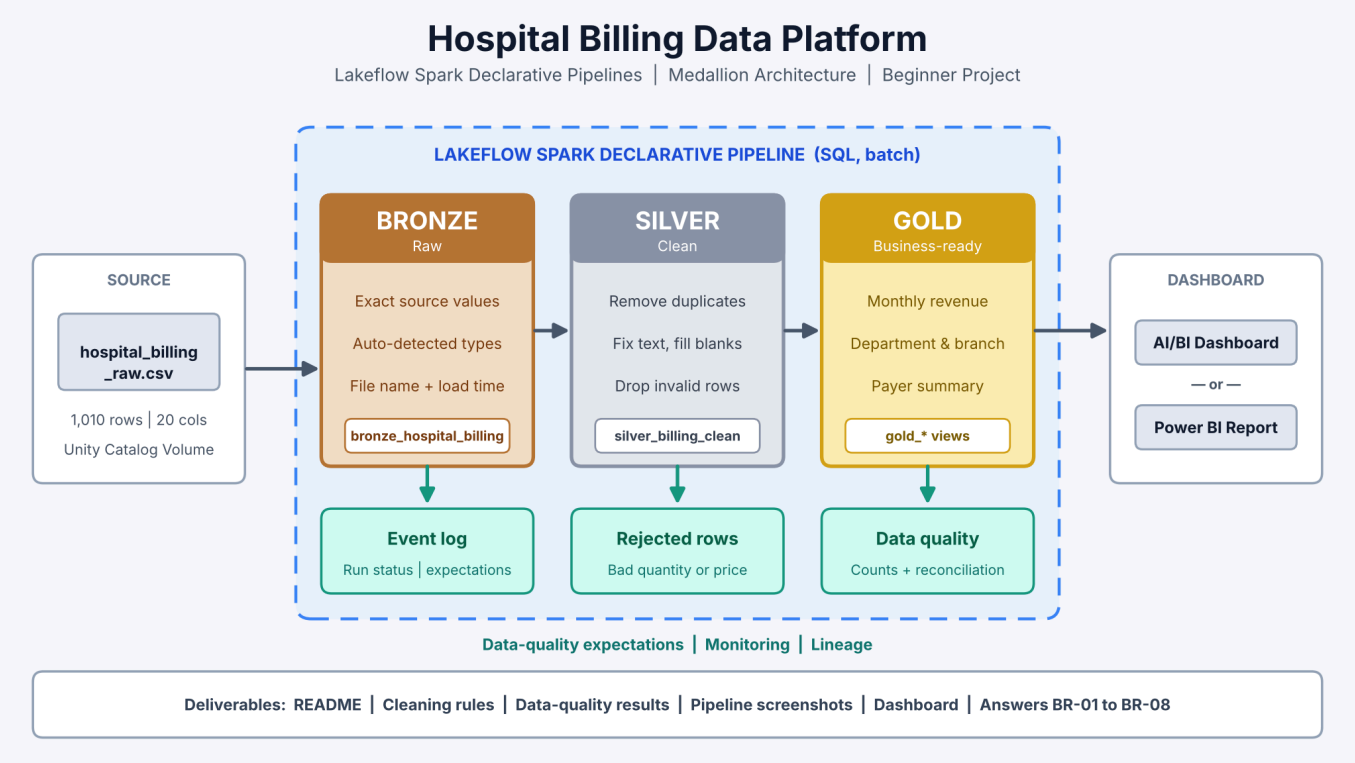
INNOMATICS RESEARCH LABS

Lakeflow Spark Declarative Pipelines

Hospital Billing Analytics Platform



Project level: Beginner • Estimated time: 6–8 hours • SQL you need: SELECT, WHERE, GROUP BY, COUNT, SUM, DISTINCT, TRIM, INITCAP, COALESCE, CASE WHEN.

1. Executive Summary

A hospital chain exports its patient billing data to a CSV file and wants simple, reliable reports on revenue, departments, branches and insurance payers. The file has common problems: extra spaces, inconsistent capital letters, blank values, duplicate rows, and a few impossible quantities and prices. You will build a Databricks Medallion Architecture that keeps the raw file, cleans it step by step, creates business-ready summary tables, and shows them in a dashboard.

Project goal: Transform a raw hospital billing CSV into trusted Gold-layer tables using Databricks SQL and Lakeflow Spark Declarative Pipelines, then build one dashboard on top of them.

2. Business Background

The hospital has six branches in three regions of India. Patients come as Outpatient, Inpatient or Emergency cases. Each visit creates one or more bills, for example a consultation, a blood test, a scan, medicines or room charges. A bill is paid by the patient (Self Pay), a private insurer, a government scheme or a corporate tie-up. Managers want to know how much money each department, branch and payer brings in.

Current business challenges

- Department, service category, payer type and city names have extra spaces and mixed capital letters.
- Some rows are missing the department, payer type, patient ID or total amount.
- Some rows appear twice, which makes revenue look bigger than it is.
- A few rows have a zero or negative quantity, or a negative price.
- Reports are made by hand in Excel every month, which is slow and error-prone.

3. Business Problem and Objectives

The hospital has no automatic way to turn the raw billing file into numbers it can trust. This project builds a repeatable pipeline that keeps the original file, cleans the data, prepares summary tables and shows them in a dashboard.

Project objectives

- Load the billing CSV into Databricks from a Unity Catalog Volume.
- Keep the raw data in a Bronze table.
- Clean the data in a Silver table and keep the bad rows in a separate rejected table.
- Create Gold summary tables for monthly revenue, departments, branches and payers.
- Count the data problems and check that no row was lost.
- Build one dashboard in Databricks AI/BI or Power BI.

4. Dataset Overview

The learning dataset hospital_billing_raw.csv is synthetic (made up for this project). It has 1,010 rows and 20 columns covering 330 patient visits in 2025. It looks like a real hospital export but contains no real patient data.

Hospital Billing Dataset :- Click Here to Download Hospital Billing Dataset

Dataset attribute	Description	Why it matters
Grain	One row = one bill line. bill_id is unique for each real bill line.	Tells you how to count rows and find duplicates
Visit & patient fields	visit_id, patient_id, gender, age, visit_date, visit_type	Supports monthly and patient analysis
Hospital fields	department, doctor_name, service_category, service_name, branch, city, region	Supports department and branch reports
Money fields	quantity, unit_price, total_amount, insurance_amount, payment_status	Supports revenue and payer reports

Dataset attribute	Description	Why it matters
Planted problems	Extra spaces, mixed case, blanks, duplicate rows, zero/negative quantity, negative price	Gives you real cleaning practice

Column dictionary

Column	Example	Description
bill_id	BL100001	Unique ID of one bill line
visit_id	VS50001	One visit can have several bill lines
patient_id	PT1234	Patient ID (sometimes blank)
patient_gender	Female	Male or Female
patient_age	34	Age in years
visit_date	2025-03-14	Date of the visit
visit_type	Outpatient	Outpatient, Inpatient or Emergency
department	Cardiology	Hospital department (sometimes blank or messy)
doctor_name	Dr. Rao	Treating doctor
service_category	Lab Test	Consultation, Lab Test, Scan, Procedure, Pharmacy, Room Charges
service_name	Blood Test	Name of the service
hospital_branch	HYD-Central	Branch code
city	Hyderabad	Branch city (sometimes messy)
region	South	South, West or North
payer_type	Private Insurance	Self Pay, Private Insurance, Government Scheme, Corporate
quantity	1	Units billed (bed-days for room charges)
unit_price	850.00	Price of one unit in ₹
total_amount	850.00	quantity × unit_price (sometimes blank)
insurance_amount	680.00	Part paid by the insurer
payment_status	Paid	Paid or Pending

Important: bill_id should be unique. If the same bill_id appears more than once, the extra rows are duplicates. visit_id is NOT unique, because one visit has several bills.

5. Key Terms for Beginners

Term	Simple meaning
Unity Catalog Volume	A folder inside Databricks where you upload files such as CSVs.
Medallion Architecture	Organising data in three layers: Bronze (raw), Silver (clean), Gold (summaries).
Lakeflow Spark Declarative Pipelines	A Databricks feature where you write SQL that says what each table should contain; Databricks works out the order and runs it.
Streaming table	A table that reads new files incrementally. We use it for Bronze.
Materialized view	A saved query result that the pipeline keeps up to date. We use it for Silver and Gold.
Expectation	A data-quality rule, e.g. quantity > 0. With ON VIOLATION DROP ROW, bad rows are removed and counted.
Reconciliation	Checking that every input row is accounted for: kept, rejected or removed as a duplicate.

6. Proposed Architecture and Data Flow

Bronze keeps the raw rows. Silver cleans them: good rows go to silver_billing_clean and bad rows go to silver_billing_rejected. Gold builds summary tables from the clean rows only, plus one data-quality table.

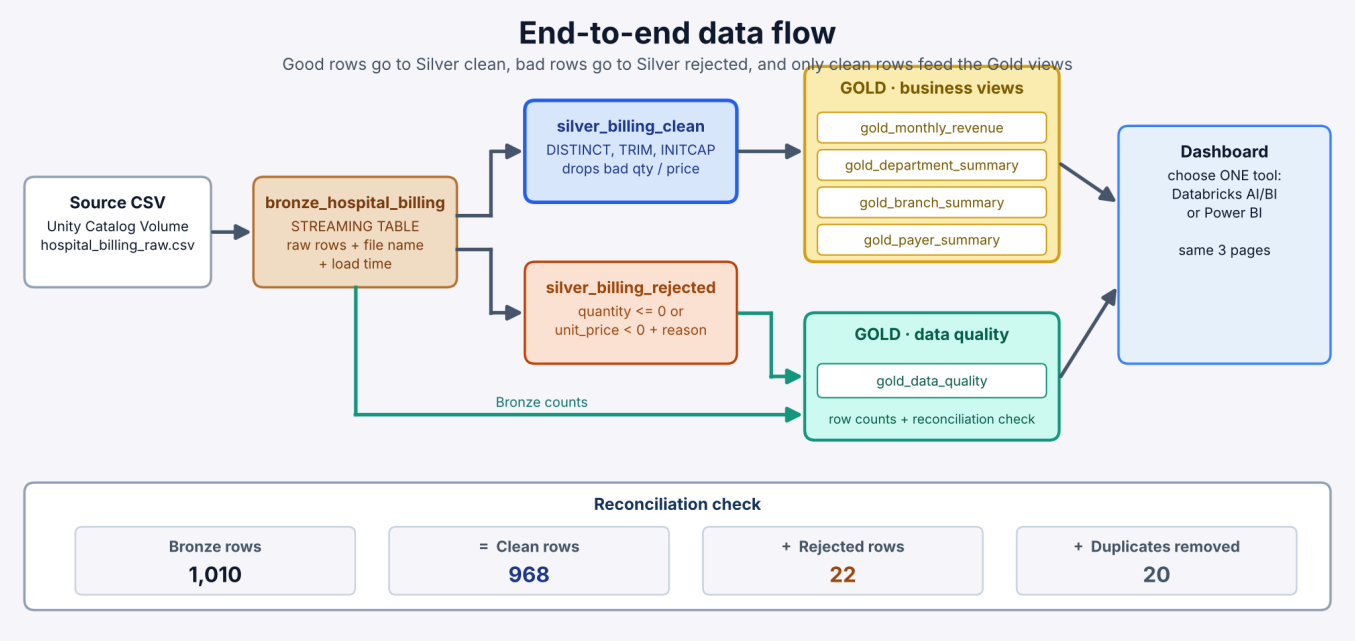


Figure 1 – End-to-end data flow and reconciliation check

7. Medallion Layer Responsibilities

Layer	Purpose	What you do	Result
Bronze	Keep the source data	Load the CSV as it is; add file name and load time	1 raw table

Layer	Purpose	What you do	Result
Silver	Clean the data	Remove duplicates, fix text, fill blanks, drop bad rows (and keep them in a rejected table)	1 clean table + 1 rejected table
Gold	Answer business questions	GROUP BY month, department, branch and payer; count data problems	5 summary tables
Dashboard	Show the results	Build 3 pages in Databricks AI/BI or Power BI	1 dashboard

8. Business Requirements

ID	Requirement	Business expectation
BR-01	Monthly revenue	Show total revenue, insurance amount and visits for each month.
BR-02	Department performance	Compare revenue, visits and average revenue per visit by department.
BR-03	Branch performance	Compare revenue and visits by region, city and branch.
BR-04	Payer analysis	Compare revenue, insurance amount and pending amount by payer type.
BR-05	Data-quality visibility	Show how many rows had each problem (duplicates, bad quantity, bad price, blanks).
BR-06	Reconciliation	Prove that Bronze rows = clean rows + rejected rows + duplicates removed.
BR-07	Repeatability	Run everything as a Databricks pipeline that can be re-run with one click.
BR-08	Dashboard	Build one dashboard (Databricks AI/BI or Power BI) on the Gold tables.

9. Data Cleaning Rules

Silver cleaning is done in four simple steps. Each step uses one SQL idea.

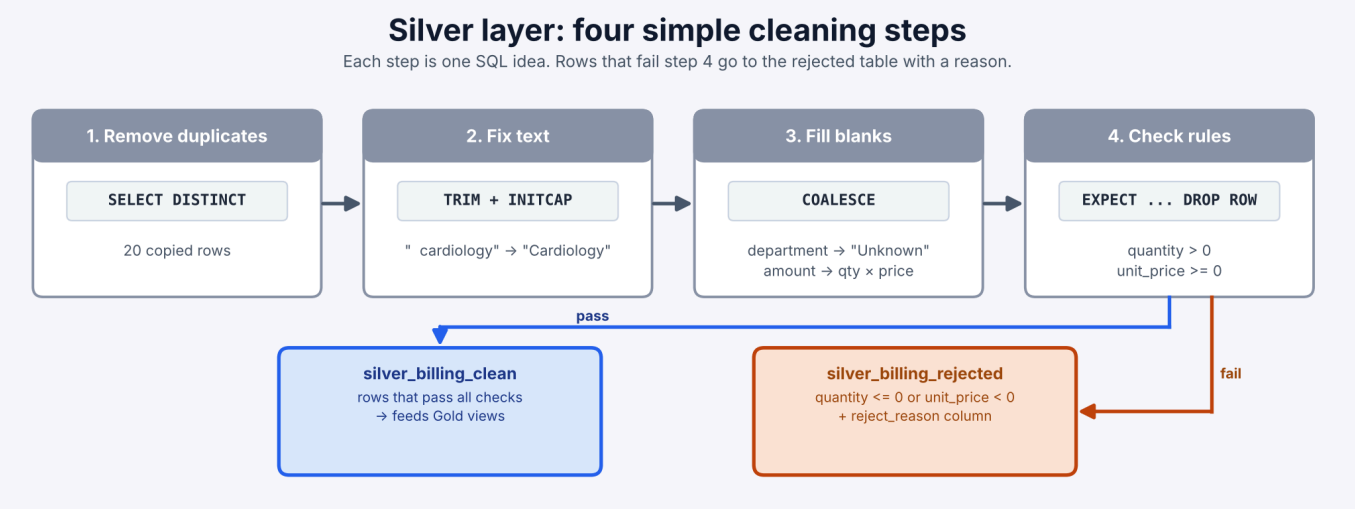


Figure 2 – The four Silver cleaning steps

Problem	Example	What to do	SQL hint
Duplicate rows	Same bill_id appears twice	Keep only one copy	SELECT DISTINCT
Extra spaces / mixed case	" cardiology", "PRIVATE INSURANCE"	Remove spaces, use Title Case	INITCAP(TRIM(col))
Blank department or payer	department is empty	Replace with "Unknown"	COALESCE(col, 'Unknown')
Blank total amount	total_amount is empty	Calculate quantity × unit_price	COALESCE(total_amount, quantity * unit_price)
Blank patient ID	patient_id is empty	Keep the row (revenue is still valid)	No change
Bad quantity or price	quantity = 0 or -1; unit_price = -850	Remove from clean table, keep in rejected table	EXPECT (quantity > 0) ON VIOLATION DROP ROW

10. Project Scope

Included

- One synthetic hospital billing CSV file.
- Batch loading from a Unity Catalog Volume.
- Bronze, Silver and Gold tables in one Lakeflow pipeline written in SQL.
- Simple data-quality checks and a reconciliation check.
- One dashboard in Databricks AI/BI or Power BI.

Not included

- Streaming or real-time data.
- More than one source file or system.
- Real patient data and privacy controls.
- Advanced data modelling (dimensions, SCD) and production alerting.

Scope note This is a learning project. Keep it simple, make it work end to end, and explain your choices.

11. Implementation Workflow

01 • Understand	Read the business questions and the 20 columns. Open the CSV and spot the problems yourself.
02 • Prepare	Create a catalog, schema and Volume. Upload the CSV.
03 • Ingest	Create the Bronze streaming table and check that 1,010 rows were loaded.
04 • Clean	Create the Silver clean and rejected tables using the four cleaning steps.
05 • Curate	Create the four Gold business tables with GROUP BY.

06 • Validate	Create the Gold data-quality table and check the reconciliation.
07 • Publish & Visualize	Build one dashboard: Databricks AI/BI or Power BI.
08 • Explain	Present your pipeline, cleaning rules, results and dashboard.

Step-by-step guide with SQL hints

Use these starter snippets. Replace or complete the parts marked with "...".

Step 02 – Prepare the storage

```
CREATE CATALOG IF NOT EXISTS healthcare_catalog;   -- or use a catalog you already have
CREATE SCHEMA  IF NOT EXISTS healthcare_catalog.billing;
CREATE VOLUME  IF NOT EXISTS healthcare_catalog.billing.raw_files;

-- Upload hospital_billing_raw.csv into the volume (Catalog Explorer → raw_files → Upload)
-- Then preview it:
SELECT * FROM read_files('/Volumes/healthcare_catalog/billing/raw_files/',
                         format => 'csv', header => true) LIMIT 10;
```

Step 03 – Create the pipeline and the Bronze table

Create a new ETL pipeline (Lakeflow Spark Declarative Pipelines), set the default catalog to healthcare_catalog and schema to billing, and add a SQL file with:

```
CREATE OR REFRESH STREAMING TABLE bronze_hospital_billing AS
SELECT *,
       _metadata.file_name AS source_file,
       current_timestamp() AS loaded_at
FROM STREAM read_files('/Volumes/healthcare_catalog/billing/raw_files/',
                       format => 'csv', header => true);
```

Step 04 – Silver clean and rejected tables

```
CREATE OR REFRESH MATERIALIZED VIEW silver_billing_clean (
  CONSTRAINT valid_quantity   EXPECT (quantity > 0)    ON VIOLATION DROP ROW,
  CONSTRAINT valid_unit_price EXPECT (unit_price >= 0) ON VIOLATION DROP ROW
) AS
SELECT DISTINCT
  bill_id, visit_id, patient_id,
  COALESCE(INITCAP(TRIM(department)), 'Unknown') AS department,
  INITCAP(TRIM(service_category))                AS service_category,
  COALESCE(total_amount, quantity * unit_price)  AS total_amount,
  ...   -- add the other columns, cleaning city and payer_type the same way
FROM bronze_hospital_billing;

CREATE OR REFRESH MATERIALIZED VIEW silver_billing_rejected AS
SELECT *,
  CASE WHEN quantity <= 0 THEN 'Quantity is zero or negative'
       ELSE 'Unit price is negative' END AS reject_reason
FROM bronze_hospital_billing
WHERE quantity <= 0 OR unit_price < 0;
```

Step 05 – Gold business tables

```
CREATE OR REFRESH MATERIALIZED VIEW gold_monthly_revenue AS
SELECT DATE_TRUNC('MONTH', visit_date) AS visit_month,
       COUNT(DISTINCT visit_id)         AS visits,
       SUM(total_amount)                AS total_revenue,
       SUM(insurance_amount)            AS insurance_amount
FROM silver_billing_clean
GROUP BY DATE_TRUNC('MONTH', visit_date);
```

Build the other three Gold tables with the same pattern:

Gold table	GROUP BY	Columns to calculate
gold_department_summary	department	visits, bills, total_revenue, avg_revenue_per_visit
gold_branch_summary	region, city, hospital_branch	visits, total_revenue
gold_payer_summary	payer_type	visits, total_revenue, insurance_amount, pending_amount (SUM with CASE WHEN payment_status = 'Pending')

Step 06 – Data quality and reconciliation

```
CREATE OR REFRESH MATERIALIZED VIEW gold_data_quality AS
SELECT COUNT(*)                                          AS bronze_rows,
       COUNT(*) - COUNT(DISTINCT bill_id)                AS duplicate_rows,
       SUM(CASE WHEN quantity <= 0 THEN 1 ELSE 0 END)    AS invalid_quantity_rows,
       SUM(CASE WHEN department IS NULL THEN 1 ELSE 0 END)  AS missing_department_rows,
       ...   -- add negative price, missing payer, missing amount
       (SELECT COUNT(*) FROM silver_billing_clean)       AS clean_rows,
       (SELECT COUNT(*) FROM silver_billing_rejected)    AS rejected_rows
FROM bronze_hospital_billing;
```

Then run the pipeline (Start), open the pipeline graph, and check: **bronze_rows = clean_rows + rejected_rows + duplicate_rows**.

12. Dashboard (Choose Databricks AI/BI or Power BI)

The dashboard is the last stage of the project. It reads only the Gold tables. Build it in **one** tool only: Option A or Option B. Both give the same three pages.

Choose one option: If you build the dashboard in Power BI, skip Option A. If you build it as a Databricks AI/BI dashboard, skip Option B.

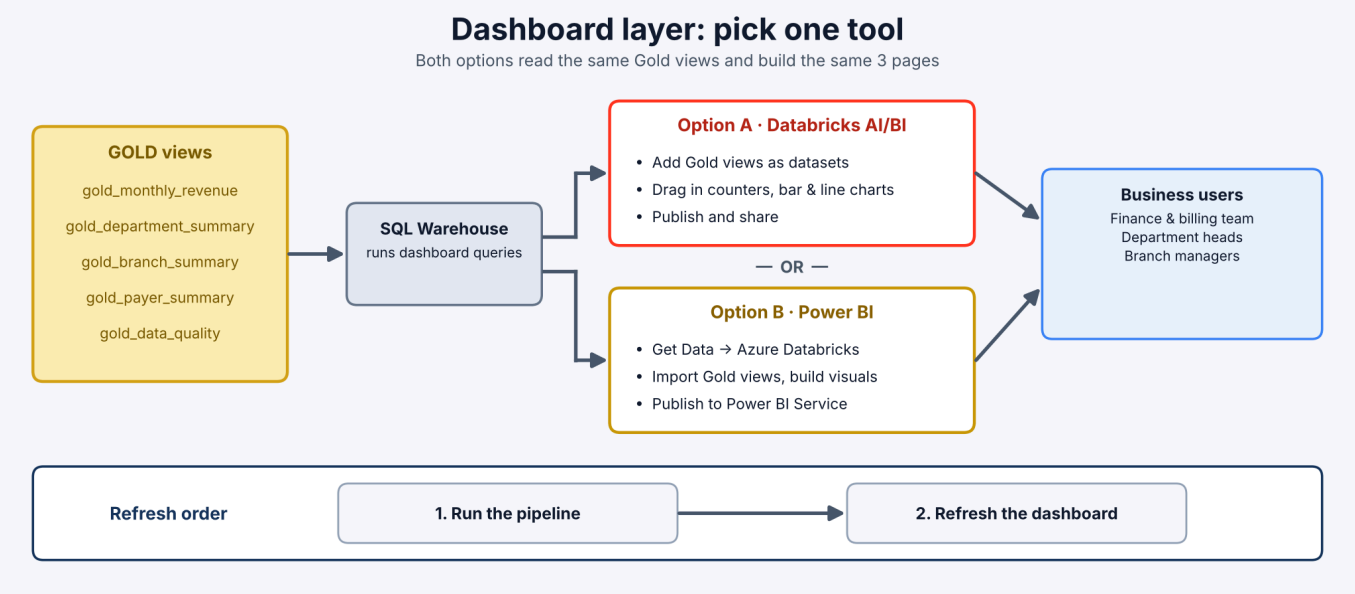


Figure 3 – Dashboard layer: one tool, same Gold tables

Dashboard pages (same for both options)

Page	Visuals	Gold table
1. Overview	KPI cards: Total Revenue, Visits, Insurance Covered, Pending Amount; line chart of revenue by month; bar chart of revenue by department	gold_monthly_revenue, gold_department_summary, gold_payer_summary
2. Branch & Payer	Bar chart of revenue by branch or region; bar chart of revenue vs insurance by payer; table of payers	gold_branch_summary, gold_payer_summary
3. Data Quality	Cards: Bronze rows, clean rows, rejected rows, duplicates; reconciliation passed (Yes/No); bar chart of problem counts	gold_data_quality

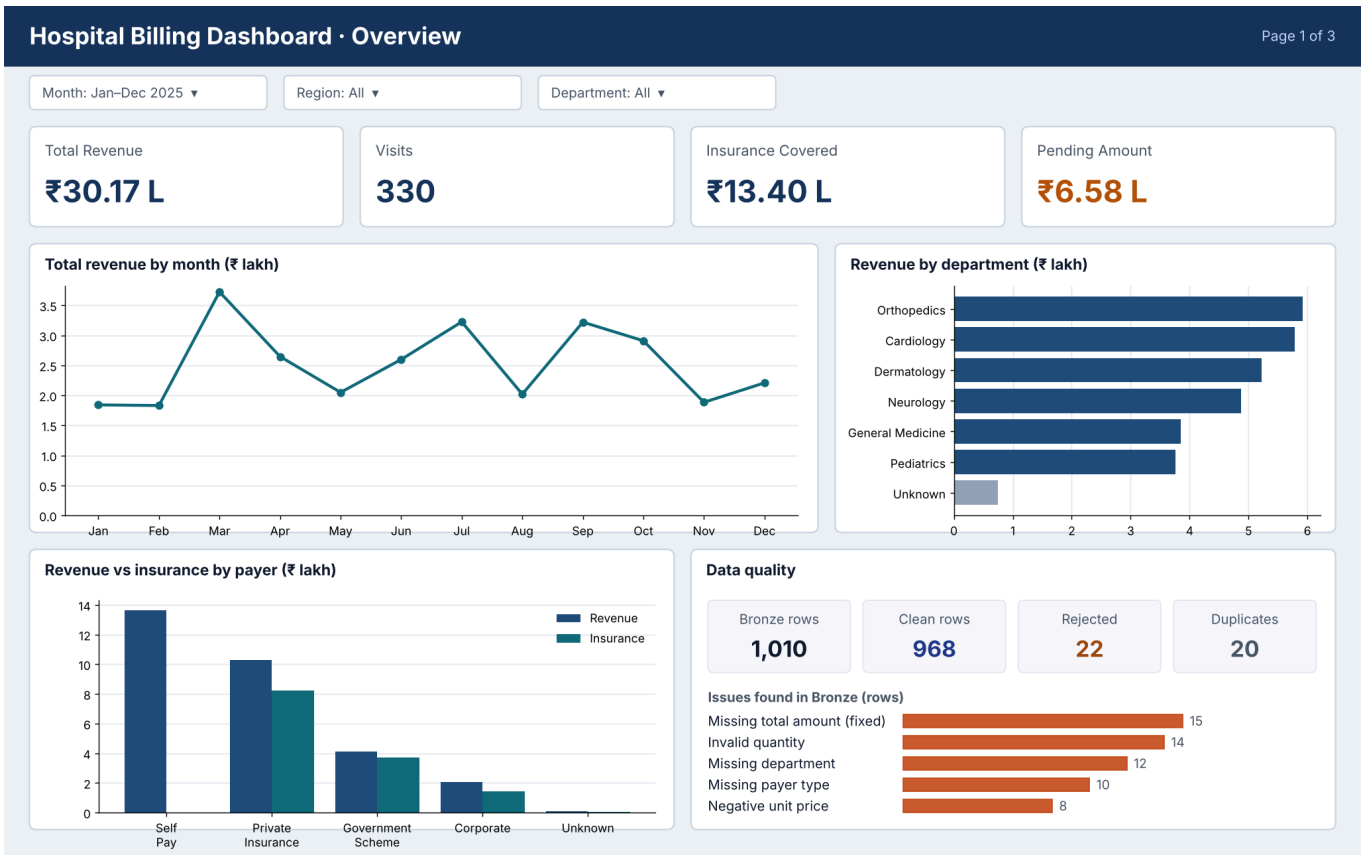


Figure 4 – Example Overview page (numbers from the reference solution)

Option A – Databricks AI/BI Dashboard

- **Create:** in Databricks choose New → Dashboard.
- **Add data:** on the Data tab, add each Gold table as a dataset (pick the table or write SELECT * FROM ...).
- **Add visuals:** on the canvas, add Counter widgets for the KPIs, a Line chart for monthly revenue and Bar charts for department, branch and payer.
- **Add filters:** add a filter for month, region or department where those columns exist.
- **Publish:** click Publish and share the dashboard with your team. Optionally add a refresh schedule.

Option B – Power BI Report

- **Connect:** in Power BI Desktop choose Get Data → Azure Databricks. Copy the Server hostname and HTTP path from your SQL warehouse (Connection details tab) and sign in.
- **Load data:** open healthcare_catalog → billing, tick the five gold_* tables, and choose Import.
- **Add visuals:** use Card visuals for the KPIs (Sum of total_revenue, visits, insurance_amount, pending_amount), a Line chart for monthly revenue and Bar charts for department, branch and payer.
- **Add filters:** add Slicers for month, region or department on the pages where those columns exist.
- **Publish:** click Publish to send the report to Power BI Service and share it. Optionally set a scheduled refresh.

Check your numbers: The Total Revenue card should match the total_revenue of all months in gold_monthly_revenue. If it does not, check your visual settings.

13. Expected Deliverables

- A pipeline with Bronze, Silver (clean + rejected) and Gold tables.
- A short list of your cleaning rules and why you chose them.
- The gold_data_quality result showing the reconciliation check passed.
- One dashboard with three pages (Databricks AI/BI or Power BI).
- A README explaining how to set up and run the project.
- Screenshots of the pipeline graph, the Gold tables and the dashboard.

14. Acceptance Criteria / Definition of Done

- The CSV loads from the Volume and Bronze has 1,010 rows.
- Silver clean has no duplicates, no extra spaces, no blank department or payer, and no bad quantity or price.
- Rejected rows are saved with a reason.
- The four Gold business tables answer BR-01 to BR-04.
- gold_data_quality shows that bronze rows = clean + rejected + duplicates.
- The pipeline runs again with one click and gives the same result.
- The dashboard shows the three pages and its numbers match the Gold tables.

15. Questions for Project Review

1. Why do we keep the raw data in Bronze instead of cleaning the file directly?
2. What is the difference between removing a row and putting it in the rejected table?
3. Why can bill_id find duplicates but visit_id cannot?
4. Why do we use TRIM before INITCAP?
5. Why is it OK to fill a blank total_amount with quantity × unit_price?
6. How does the reconciliation check prove that no row was lost?
7. Why does the dashboard read Gold tables instead of Bronze?
8. Which dashboard tool did you choose, and why?

16. Final Business Outcome

At the end of the project, the hospital has a repeatable pipeline that turns a messy billing file into clean, trusted summary tables and a simple dashboard. You can explain where the data came from, what problems you found, how you fixed them, and how you proved the numbers are correct.

Key takeaway Medallion Architecture means each layer makes the data a little more trustworthy: Bronze keeps it, Silver cleans it, Gold summarises it, and the dashboard shares it.